

```
1
2 var gameConfig = {
3   'duration': 30000,
4   'presentation': false,
5   'ghost': false,
6   'ticker': false,
7   'gameid': false,
8   'stats': true,
9   'alwaysUseServer': false,
10  'autoplay': false,
11  'playable': true,
12  'logging': false,
13  'replay': false,
14  'saveServer': 'chartGame',
15  'readServer': 'chartGame',
16  'readServerLimit': 1000
17 }
18 // in cima a main.js, dopo le altre variabili
19 let titoliRend = []; // rendimento di ogni titolo
20 let sp500Rend = []; // rendimento relativo all'S&P500
21 // all config vars are configurable by query string on localhost;
22 // otherwise only ticker, gameid, and replay
23 if(window.location.hostname === 'localhost') {
24   Object.keys(gameConfig).forEach(function(key) {
25     var q = getQueryVariable(key);
26     if(q === 'true') {
27       gameConfig[key] = true;
28     } else if(q === 'false') {
29       gameConfig[key] = false;
30     } else if(q) {
31       gameConfig[key] = q;
32     }
33   })
34 } else {
35   gameConfig.ticker = getQueryVariable('ticker');
36   gameConfig.gameid = getQueryVariable('gameid');
37   gameConfig.replay = getQueryVariable('replay') === 'true' ? true :
... false;
38 }
39 var isTerminal = false;
40 if(gameConfig.replay) {
41   gameConfig.duration = 15000;
42   gameConfig.presentation = true;
43   gameConfig.ghost = true;
44   gameConfig.playable = false;
45 }
46
47 d3.select("body").classed("presentation", gameConfig.presentation);
48
49 // set of levels available
50 var tickers = [
```

```
51  {
52    "ticker": "aapl",
53    "name": "APPLE US Equity"
54  },
55  {
56    "ticker": "dell",
57    "name": "DELL US Equity"
58  }/*,
59  {
60    "ticker": "cat",
61    "name": "CATERPILLAR US Equity"
62  },
63  {
64    "ticker": "coke",
65    "name": "COCA-COLA US Equity"
66  },
67  {
68    "ticker": "brkb",
69    "name": "BERKSHIRE HATHAWAY US Equity"
70  },
71  {
72    "ticker": "ene",
73    "name": "ENRON US Equity"
74  }
75
76  /*,
77  {
78    "ticker": "nflx",
79    "name": "NFLX US Equity"
80  },
81  {
82    "ticker": "tsla",
83    "name": "TSLA US Equity"
84  },
85  {
86    "ticker": "brkb",
87    "name": "BRK/B US Equity"
88  },
89  {
90    "ticker": "mcz",
91    "name": "MCZ US Equity"
92  },
93  {
94    "ticker": "pg",
95    "name": "PG US Equity"
96  },
97  {
98    "ticker": "ene",
99    "name": "Enron US Equity"
100 },
101 {
```

```
102     "ticker": "znga",
103     "name": "ZNGA US Equity"
104 },
105 {
106     "ticker": "ge",
107     "name": "GE US Equity"
108 },
109 {
110     "ticker": "coke",
111     "name": "COKE US Equity"
112 },
113 {
114     "ticker": "googl",
115     "name": "GOOGL US Equity"
116 },
117 {
118     "ticker": "cat",
119     "name": "CAT US Equity"
120 },
121
122 // second batch
123
124 {
125     "ticker": "glen",
126     "name": "GLEN LN Equity"
127 },
128 {
129     "ticker": "mcd",
130     "name": "MCD US Equity"
131 },
132 {
133     "ticker": "dd",
134     "name": "DuPont US Equity"
135 },
136 {
137     "ticker": "xom",
138     "name": "XOM US Equity"
139 },
140 {
141     "ticker": "1788",
142     "name": "1788 HK (Guotai Junan)",
143     "description": "On Nov. 23, Guotai Junan announced its CEO was
... missing and could not be reached.
144 },
145 {
146     "ticker": "cag",
147     "name": "CAG US Equity"
148 },
149 {
150     "ticker": "f",
151     "name": "F US Equity"
```

```
152     },
153     {
154         "ticker": "corn",
155         "name": "CORN US Equity"
156     },
157     {
158         "ticker": "gpro",
159         "name": "GPRO US Equity"
160     },
161     {
162         "ticker": "yhoo",
163         "name": "YH00 US Equity"
164     },
165     {
166         "ticker": "lulu",
167         "name": "LULU (Lululemon)"
168     },
169     {
170         "ticker": "hal",
171         "name": "HAL (Halliburton)"
172     },
173     {
174         "ticker": "swhc",
175         "name": "SWHC (Smith & Wesson)"
176     },
177     {
178         "ticker": "kbio",
179         "name": "KBIO (KaloBios)"
180     },
181     {
182         "ticker": "cmg",
183         "name": "CMG (Chipotle)"
184     },
185     {
186         "ticker": "bks",
187         "name": "BKS (Barnes & Noble)"
188     },
189     {
190         "ticker": "gs",
191         "name": "GS (Goldman Sachs)"
192     },
193     {
194         "ticker": "dis",
195         "name": "DIS (Disney)"
196     },
197
198     {
199         "ticker": "fed",
200         "name": "SPX (Fed liftoff)"
201     },
202
```

```
203 {
204   "ticker": "hsi",
205   "name": "HSI (Hang Seng Index)"
206 },
207 {
208   "ticker": "shcomp",
209   "name": "SHCOMP (Shanghai Composite)"
210 }
211 */
212 ];
213
214 // UNUSED
215 /*
216 {
217   "ticker": "ibm", // shitty
218   "name": "IBM US Equity"
219 },
220 {
221   "ticker": "jpm",
222   "name": "JPM US Equity"
223 },
224 {
225   "ticker": "rtn",
226   "name": "RTN US Equity",
227   "description": "The previous day, SpaceX's rocket blew up. Not
... that it really mattered to Raytheon."
228 },
229 {
230   "ticker": "grpn", // way too flat intraday
231   "name": "GRPN US Equity"
232 },
233 {
234   "ticker": "7974",
235   "name": "7974 JT Equity (Nintendo)",
236   "description":
... "https://twitter.com/DavidInglesTV/status/659549109410332672"
237 },
238 */
239
240 tickers = _.shuffle(tickers);
241
242 var cash = 500;
243 var cashLiquid = true;
244 var levelNumber = 0;
245
246 var userId = Math.random().toString().split(".")[1];
247 var userHistory = {};
248
249 // persist userId and cash and userHistory
250 if(typeof(Storage) !== "undefined") {
251   if(!localStorage.getItem("userId")) {
```

```
252     try {
253         localStorage.setItem("userId", userId);
254     } catch(e) {
255         console.error(e);
256     }
257 } else {
258     userId = localStorage.getItem("userId");
259 }
260
261 if(!localStorage.getItem("cash")) {
262     try {
263         localStorage.setItem("cash", cash);
264     } catch(e) {
265         console.error(e);
266     }
267 } else {
268     cash = localStorage.getItem("cash");
269 }
270
271 if(!localStorage.getItem("userHistory")) {
272     try {
273         localStorage.setItem("userHistory",
... JSON.stringify(userHistory));
274     } catch(e) {
275         console.error(e);
276     }
277 } else {
278     userHistory = JSON.parse(localStorage.getItem("userHistory"));
279 }
280 }
281 window.onunload = window.onbeforeunload = function() {
282     localStorage.setItem("cash", cash);
283     localStorage.setItem("userHistory", JSON.stringify(userHistory));
284 };
285
286 var indexFund;
287
288 var analysisStats = [];
289 var cashRules = [];
290
291 var tips = [
292     "Premi e tieni premuto in qualsiasi punto sul grafico per
... comprare.",
293     "Mantieni premuto per mantenere la posizione del titolo e rilascia
... per vendere.",
294     "Il trascinamento non ha alcun effetto. L'unico controllo è quello
... del tempo."
295 ];
296
297 var taglines = [
298     "Buy Low Sell High",
```

```
299     "So You Think You Can Trade?",
300     "America's Next Top Trader",
301     "Portrait of the Chartist as a Young Man",
302     "Get Rich Quick!",
303     "Terminal Professional Service Lite",
304     "Crushing Stock Saga",
305     "Flappy Stock",
306     "Uncharted 5",
307     "li'l charts",
308     "The Big Long"
309 ];
310
311 // initCanvas();
312
313 // gets s&p, then calls init
314 getIndexFund();
315
316 function getIndexFund() {
317     d3.csv("spx.csv", function(error, data) {
318
319         // more minimal version of processData below...
320         var parseDate = d3.time.format("%x").parse;
321         data.forEach(function(d) {
322             d.date = parseDate(d.date);
323             d.price = +d.price;
324         });
325         data.sort(function(a,b) {
326             return a.date - b.date;
327         });
328         indexFund = {
329             "ticker": "spx",
330             "name": "SPX",
331             "values": data
332         }
333
334         // ready to go
335         init();
336     });
337 }
338
339 function init() {
340
341     var isTerminal = false;
342     // find ticker specified in hash, if any
343     var tickerIndex = tickers.map(function(d) { return d.ticker;
... }).indexOf(gameConfig.ticker);
344
345     var buttonText = gameConfig.replay ? "Watch" : "Play";
346
347     // FED LIFTOFF HACK
348     if(gameConfig.ticker) buttonText += " " +
```

```
348... gameConfig.ticker.toUpperCase()
349    //if(gameConfig.ticker && tickerIndex > -1) buttonText += " " +
...    tickers[tickerIndex].name;
350
351    if(gameConfig.gameid) buttonText += " challenge";
352    window.dvzqueue = queue;
353    d3.select(".opener")
354        .select("button")
355        .text(buttonText)
356        .on("click", getNextLevel)
357        .on("touchstart", function(d) { d3.select(this).classed("hover",
... true); })
358        .on("touchend", function(d) { d3.select(this).classed("hover",
... false); });
359
360    d3.select(".opener h3").text(_.sample(taglines));
361
362    if(gameConfig.autoplay) getNextLevel();
363 }
364
365 function getNextLevel() {
366     levelNumber++;
367
368     // this function is only called by "next" buttons, so... loading
... indicators!
369     d3.select(this).text("...");
370
371     d3.select('#full-header').classed('hidden', true);
372
373     // find ticker specified in hash, if any
374     var tickerIndex = tickers.map(function(d) { return d.ticker;
... }).indexOf(gameConfig.ticker);
375
376     if(gameConfig.ticker && tickerIndex !== -1) {
377         // load level specified in hash
378         loadTicker(tickers.splice(tickerIndex,1)[0]);
379     }
380     //else if(!isTerminal && innerWidth <= 1060 && levelNumber % 6 == 3)
... {
381         // loadAd();
382         // }
383     else if(tickers.length) {
384         loadTicker(tickers.pop());
385     } else {
386         // no more levels
387         gameOver();
388     }
389 }
390
391 // load ad
392 function loadAd() {
```



```
393
394     var i = d3.select(".levels").selectAll(".ad-item").size();
395
396     var adWrapper = d3.select(".levels")
397         .append("div.ad-item")
398         .style("z-index", d3.selectAll(".levels > div").size());
399
400     adWrapper.append("div.toolbar")
401         .append("button.next")
402         .text("Next")
403         .on("click", getNextLevel)
404         .on("touchstart", function(d) { d3.select(this).classed("hover",
... true); })
405         .on("touchend", function(d) { d3.select(this).classed("hover",
... false); });
406
407     adWrapper.append("p").text("Advertisement");
408
409     var ad = adWrapper.append("div.ad");
410
411     // get intended size from data attribute
412     // e.g. <div class="bannerad-sized" data-size="728,90"></div>
413     // fall back to widest IAB size that fits container
414     var size = ad.node().dataset.size ?
... ad.node().dataset.size.split(",") : getBestAdSize(ad.node());
415
416
417     ad.node().style.display = "block";
418     var randValue = new String(Math.random()).substring(2,11);
419     var n = i + 1;
420     ad.node().innerHTML = new_leader + '&position=box' + n + '&ord=' +
... randValue + "'></iframe>';
421
422 }
423
424 // load file
425 function loadTicker(ticker) {
426     var qu = window.dvzqueue();
427     qu.defer(d3.csv, ticker.ticker + ".csv");
428
429     if(gameConfig.ghost) {
430         qu.defer(getTrades, ticker.ticker, undefined);
431     } else if(ticker.ticker === gameConfig.ticker && gameConfig.gameid)
... {
432         qu.defer(getTrades, ticker.ticker, gameConfig.gameid);
433     }
434
435     qu.awaitAll(function(err, data) {
436
437         var stock = processData(ticker, data[0]);
438
```

```
439     if(data.length > 1) {
440         stock.competitor = data[1];
441         stock.competitor.trades.forEach(function(d,i) {
442             d.final = d[1];
443         });
444         stock.competitor.gameid = gameConfig.ghost ? "ghost" :
... +gameConfig.gameid;
445     }
446
447     /// uhhhhh "GARBAGE COLLECTION" : )
448     d3.selectAll(".levels .chart-item").remove();
449
450     d3.select(".levels")
451         .style("z-index", "2")
452         .append("div.chart-item")
453         .style("z-index", d3.selectAll(".levels > div").size())
454         .datum(stock)
455         .call(playLevel);
456 });
457
458 }
459
460 // process data
461 function processData(stock, data) {
462
463     // console.log(stock.ticker);
464
465     // normalize length
466     data = data.slice(0,500);
467
468     // most are mm/dd/yyyy...
469     var parseDate = d3.time.format("%x").parse;
470     // but, uh, some are mm/dd/yyyy hh:mm:ss (24-hr) :-/
471     if(data[0].date.split(" ").length == 2) {
472         parseDate = d3.time.format("%x %X").parse;
473     }
474
475     data.forEach(function(d) {
476         d.date = parseDate(d.date);
477         d.price = +d.price;
478     });
479
480     data.sort(function(a,b) {
481         return a.date - b.date;
482     });
483
484     data.forEach(getStdev, data);
485     // data.forEach(getStdev2, data);
486
487     var t0 = _.min(data, f('date'));
488     var t1 = _.max(data, f('date'));
```

```
489     var stockReturn = (t1.price - t0.price) / t0.price;
490
491     if(indexFund) {
492         var spxt0 = closest(f('date'))(indexFund.values, t0.date);
493         var spxt1 = closest(f('date'))(indexFund.values, t1.date);
494         var indexReturn = (spxt1.price - spxt0.price) / spxt0.price;
495     }
496
497     return {
498         "ticker": stock.ticker,
499         "name": stock.name,
500         "values": data,
501         "getSmoothDomain": getSmoothDomainFunction(data),
502         "period": Math.min(data[1].date - data[0].date, data[2].date -
... data[1].date),
503         "trades": [],
504         "cash": [],
505         "stockReturn": stockReturn,
506         "indexReturn": indexReturn ? indexReturn : stockReturn,
507         "traderReturn": null,
508         "sharesHeld": 0,
509         "challengeUrl": window.location.href
510     };
511 }
512
513 // render
514 function playLevel(selection) {
515     selection.each(function(stock) {
516
517         var item = d3.select(this),
518             margin = {top: 50, right: 40, bottom: 20, left: 0},
519             width = this.offsetWidth - margin.left - margin.right,
520             height = this.offsetHeight - margin.top - margin.bottom,
521             duration = gameConfig.duration,
522             timeframe = stock.period * (width / 8),
523             timeBounds,
524             priceBounds,
525             latest;
526
527         stock.cash.push({
528             "date": d3.min(stock.values, f('date')),
529             "cash": cash
530         });
531
532         // DEBUG
533         // if(gameConfig.presentation) {
534         //     margin.top = 5;
535         //     height = this.offsetHeight - margin.top - margin.bottom;
536         // }
537         // console.log(stock);
538         // console.log(stock.ticker, stock.values.length, stock.period /
```

```
538... (24 * 60 * 60 * 1000))
539    // console.log(d3.mean(stock.values.map(f('σ'))));
540    // console.log(stock.period / (24 * 60 * 60 * 1000));
541    // END DEBUG
542
543    var showTipThrottled = _.throttle(showTip, 5000, {leading:
... false});
544
545    var dollarFormat = d3.format('$,.2f');
546    var dollarFormat0 = d3.format('$,.0f');
547    var percentFormat = d3.format(".1%");
548    var percentFormat0 = d3.format(".0%");
549
550    var xTickScale = d3.scale.linear()
551        .domain([320, 1280])
552        .range([4, 10]);
553
554    var colorScale = d3.scale.linear()
555        .domain([-0.01, 0.01])
556        .range(["#ff0000", "#00ff00"])
557        .clamp(true);
558
559    var iconNumberScale = d3.scale.log()
560        .domain([1, 1000000])
561        .range([1, 10]);
562
563    var x = d3.time.scale()
564        .domain([
565            d3.min(stock.values, f('date')),
566            d3.min(stock.values, f('date'))
567        ])
568        .range([0, width]);
569
570    var y = d3.scale.linear()
571        .domain(d3.extent(stock.values, f('price')))
572        .range([height, 0]);
573
574    var tScale = d3.time.scale()
575        .domain(d3.extent(stock.values, f('date')))
576        .range([0, duration]);
577
578    var xAxis = d3.svg.axis()
579        .scale(x)
580        .ticks(xTickScale(width))
581        .orient("bottom");
582
583    var yAxis = d3.svg.axis()
584        .scale(y)
585        .orient("right")
586        .tickFormat(d3.format("$"));
587
```

```
588     var gridlines = d3.svg.axis()  
589         .scale(y)  
590         .tickSize(-width, 0)  
591         .tickValues(gridlineTicks)  
592         .orient("left");  
593  
594     var stepLine = d3.svg.line()  
595         .x(function(d) { return x(d.date); })  
596         .y(function(d) { return y(d.price); })  
597         .interpolate("step-after");  
598  
599     var stepArea = d3.svg.area()  
600         .x(function(d) { return x(d.date); })  
601         .y0(y.range()[0])  
602         .y1(function(d) { return y(d.price); })  
603         .interpolate("step-after");  
604  
605     var line = d3.svg.line()  
606         .x(function(d) { return x(d.date); })  
607         .y(function(d) { return y(d.price); });  
608  
609     // DOM-building begins here  
610  
611     var toolbar = item.append("div.toolbar");  
612     var alerts = item.append("div.alerts");  
613  
614     var timeLabel =  
615     ... toolbar.append("div.time").text(timerFormat(duration));  
616     var cashHolder = toolbar.append("div.cash");  
617     var cashLabel =  
618     ... cashHolder.append("span.net").text(dollarFormat(cash));  
619     var cashChange = cashHolder.append("div.change");  
620     var tickerLabel =  
621     ... toolbar.append("div.ticker").text(stock.name).style("display",  
622     ... "none");  
623     if(gameConfig.replay) tickerLabel.style("display", "");  
624  
625     var svg = item.append("svg")  
626         .attr("width", width + margin.left + margin.right)  
627         .attr("height", height + margin.top + margin.bottom)  
628         .append("g")  
629         .attr("transform", "translate(" + margin.left + "," +  
630     ... margin.top + ")");  
631  
632     var clipPath = svg.append("clipPath")  
633         .attr("id", "plot-area-clip")  
634         .append("rect")  
635         .attr("x", 0)  
636         .attr("y", 0)  
637         .attr("width", width)  
638         .attr("height", height);
```

```
634
635     var gridlinesG = svg.append("g")
636         .attr("class", "gridlines axis")
637         .call(gridlines);
638
639     var xAxisG = svg.append("g")
640         .attr("class", "x axis")
641         .attr("transform", "translate(0," + height + ")")
642         .call(xAxis);
643
644     var yAxisG = svg.append("g")
645         .attr("class", "y axis")
646         .attr("transform", "translate(" + width + "," + 0 + ")")
647         .call(yAxis);
648
649     var lineArea = svg.append("path")
650         .datum(stock.values)
651         .attr("class", "area")
652         .attr("clip-path", "url(#plot-area-clip)")
653         .attr("d", stepArea);
654
655     var linePath = svg.append("path")
656         .datum(stock.values)
657         .attr("class", "line share")
658         .attr("clip-path", "url(#plot-area-clip)")
659         .attr("d", stepLine);
660
661     var yMark = svg.append("g.y-mark")
662         .attr("transform", "translate(" + (width + 3) + "," + 0 +
...   ")");
663     yMark.append("path")
664         .attr("d", d3.svg.symbol().type("triangle-up").size(30))
665         .style("transform", "rotate(-90deg)")
666         .style("fill", "#fff");
667     yMark.append("image")
668         .attr("xlink:href", "certificate-white.png")
669         .attr("width", 18)
670         .attr("height", 12)
671         .attr("x", 6)
672         .attr("y", -6);
673
674     svg.append("circle.cursor-indicator")
675         .attr("cx", "50")
676         .attr("cy", "50")
677         .attr("r", 1e-6);
678
679     tick(0);
680     startLevel();
681
682     //////////////////////////////////////
683
```

```
684     function startLevel() {
685
686         item.classed("playing", true);
687         d3.select("body").classed("playing", true);
688
689         if(gameConfig.playable) {
690             var dragger = d3.behavior.drag()
691                 .on("dragstart", dragstart)
692                 .on("drag", drag)
693                 .on("dragend", dragend);
694
695             item
696                 .call(dragger)
697                 .on("mousemove.cursor", mousemove);
698         }
699
700         d3.timer(tick);
701     }
702
703     function tick(t) {
704
705         // level is over
706         if(t > tScale.range()[1]) {
707             // liquidate any remaining shares
708             dragend();
709             // unbind trade listeners
710             item.on(".drag", null);
711             item.on(".cursor", null);
712         }
713
714         // heyo adaptive feedback!
715         if(!(d3.max(stock.trades, function(d) { return tScale(d[1].date)
... - tScale(d[0].date); }) > 500)) {
716             showTipThrottled();
717         }
718
719         timeBounds = [new Date(+tScale.invert(t) - timeframe),
... tScale.invert(t)];
720         priceBounds = stock.getSmoothDomain(tScale.invert(t));
721
722         x.domain(timeBounds);
723         y.domain(priceBounds);
724
725         xAxisG.call(xAxis);
726         yAxisG.call(yAxis);
727         gridlinesG.call(gridlines);
728         linePath.attr("d", stepLine);
729         lineArea.attr("d", stepArea);
730
731         var boundedData = stock.values.filter(function(d) { return
... d.date > timeBounds[0] && d.date <= timeBounds[1]; });
```

```
732     if(boundedData.length) {
733         latest = _.max(boundedData, function(d) { return d.date; });
734     }
735     if(!cashLiquid) {
736         cash = latest.price * stock.sharesHeld;
737     }
738
739     stock.trades.filter(function(d) { return d.outstanding;
... }).forEach(function(d) {
740         d[1] = {
741             "date": timeBounds[1],
742             "price": latest.price
743         };
744     });
745
746     yMark.attr("transform", "translate(" + (width + 3) + "," +
... y(latest.price) + ")");
747
748     svg.selectAll("path.transaction.live")
749         .data(stock.trades)
750         .enter()
751         .append("path.transaction.live");
752
753     svg.selectAll("path.transaction.live")
754         .attr("d", line)
755         .classed("outstanding", f('outstanding'))
756         .style("stroke", function(d) { return colorScale( d[1].price -
... d[0].price )});
757
758     cashLabel.text(dollarFormat(cash))
759         .classed("liquid", cashLiquid);
760     timeLabel.text(timerFormat(duration-t));
761
762     if(stock.competitor) {
763
764         var ghostTrades = stock.competitor.trades
765             .slice(0,1500)
766             .filter(function(d,i) {
767                 return d[0].date < timeBounds[1];
768             });
769         ghostTrades.forEach(function(d,i) {
770             if (d.final.date < timeBounds[1]) {
771                 // use final data
772                 d[1] = d.final;
773             } else {
774                 // trade is outstanding;
775                 // use then-latest data
776                 d[1] = {
777                     "date": timeBounds[1],
778                     "price": latest.price
779                 };
```



```
780     }
781   });
782
783   svg.selectAll("path.transaction.competitor")
784     .data(ghostTrades)
785     .enter()
786     .append("path.transaction.competitor")
787     .classed("ghost", stock.competitor.trades.length > 100);
788
789   svg.selectAll("path.transaction.competitor")
790     .attr("d", line)
791     .style("stroke", function(d) { return colorScale(d[1].price
... - d[0].price); });
792   }
793
794   // level is over, pt. 2 :-/
795   if(t > tScale.range()[1]) {
796     return levelOver();
797   };
798
799 }
800
801 function dragstart() {
802
803   // stock is already being traded.
804   // sloppy hack away from per-share trading...
805   if(!cashLiquid) return;
806   cashLiquid = false;
807
808   item.classed("holding", true);
809
810   var point = d3.mouse(svg.node());
811
812   stock.sharesHeld = cash / latest.price;
813
814   var trade = [
815     {
816       "date": timeBounds[1],
817       "price": latest.price
818     },
819     {
820       "date": timeBounds[1],
821       "price": latest.price
822     }
823   ];
824   trade.outstanding = true;
825   trade.id = Math.random().toString().split(".")[1];
826
827   stock.trades.push(trade);
828   /*
829   svg.selectAll("text.dollabill.id"+trade.id)
```

```
830     .data(d3.range(Math.round(iconNumberScale(trade[0].price *
... stock.sharesHeld))))
831     .enter()
832     .append("text.dollabill.id"+trade.id)
833     .text("$")
834     .attr("x", function(d,i) { return Math.random() * 30 - 15; })
835     .attr("y", function(d,i) { return Math.random() * 30 - 15; })
836     .transition()
837     .duration(500)
838     .delay(function(d,i) { return 10*i; })
839     .ease("linear")
840     .attr("x", x(trade[0].date))
841     .attr("y", y(trade[0].price))
842     .remove();
843     svg.selectAll("image.shareicon.id"+trade.id)
844     .data(d3.range(Math.round(iconNumberScale(stock.sharesHeld))))
845     .enter()
846     .append("image.shareicon.id"+trade.id)
847     .attr("xlink:href","certificate-white.png")
848     .attr("width", 18)
849     .attr("height", 12)
850     .attr("x", function(d,i) { return x(trade[0].date) +
... (Math.random() * 30 - 15); })
851     .attr("y", function(d,i) { return y(trade[0].price) +
... (Math.random() * 30 - 15); })
852     .transition()
853     .duration(500)
854     .delay(function(d,i) { return 20*i; })
855     .ease("linear")
856     .attr("x", point[0])
857     .attr("y", point[1])
858     .remove();
859 */
860     svg.select(".cursor-indicator")
861     .attr("cx", point[0])
862     .attr("cy", point[1])
863     .transition()
864     .duration(250)
865     .attr("r", "25");
866
867 }
868
869 function drag() {
870
871     var point = d3.mouse(svg.node());
872
873     svg.select(".cursor-indicator")
874     .attr("cx", point[0])
875     .attr("cy", point[1]);
876 }
877
```

```
878     function mousemove() {
879         if(!cashLiquid) return;
880         svg.select(".cursor-indicator")
881             .attr("cx", d3.mouse(svg.node())[0])
882             .attr("cy", d3.mouse(svg.node())[1])
883             .transition()
884             .duration(250)
885             .attr("r", 5);
886     }
887
888     function dragend() {
889
890         item.classed("holding", false);
891
892         // liquidate any outstanding transactions
893         var outstandingTransactions = stock.trades.filter(function(d) {
894             return d.outstanding;
895         });
896         outstandingTransactions.forEach(function(d) {
897             d.outstanding = false;
898
899             cash = d[1].price * stock.sharesHeld;
900
901             // terrible hacky thing for zero-duration trades
902             // cf.
903             https://github.com/bizweekgraphics/stock-chart-game/issues/31
904             if(d[1].date - d[0].date) {
905                 cashChangeValue = (d[1].price - d[0].price) *
906                 stock.sharesHeld;
907                 cashChange.text(dollarFormat(cashChangeValue))
908                     .style("color", colorScale(cashChangeValue))
909                     .style("opacity", 1)
910                     .transition()
911                     .duration(1000)
912                     .style("opacity", 0);
913             }
914             /*
915             svg.selectAll("text.dollabill.id"+d.id)
916                 .data(d3.range(Math.round(iconNumberScale(d[0].price *
917                 stock.sharesHeld))))
918                 .enter()
919                 .append("text.dollabill.id"+d.id)
920                 .text("$")
921                 .attr("x", x(d[1].date))
922                 .attr("y", y(d[1].price));
923             svg.selectAll("text.dollabill.id"+d.id).transition()
924                 .duration(500)
925                 .delay(function(d,i) { return 10*i; })
926                 .ease("linear")
927                 .attr("x", function(d,i) { return Math.random() * 30 - 15;
928             })
929         })
```

```
925         .attr("y", function(d,i) { return Math.random() * 30 - 15;
...     })
926         .remove();
927     svg.selectAll("image.shareicon.id"+d.id)
928
...     .data(d3.range(Math.round(iconNumberScale(stock.sharesHeld))))
929         .enter()
930         .append("image.shareicon.id"+d.id)
931         .attr("xlink:href","certificate-white.png")
932         .attr("width", 18)
933         .attr("height", 12)
934         .attr("x", d3.select(".cursor-indicator").attr("cx"))
935         .attr("y", d3.select(".cursor-indicator").attr("cy"));
936     svg.selectAll("image.shareicon.id"+d.id).transition()
937         .duration(500)
938         .delay(function(d,i) { return 20*i; })
939         .ease("linear")
940         .attr("x", function() { return x(d[1].date) + (Math.random()
...     * 30 - 15); })
941         .attr("y", function() { return y(d[1].price) +
...     (Math.random() * 30 - 15); })
942         .remove();
943 */
944     svg.select(".cursor-indicator")
945         .transition()
946         .duration(250)
947         .attr("r", 1e-6);
948
949     stock.sharesHeld = 0;
950     cashLiquid = true;
951
952 });
953
954 }
955
956 function levelOver() {
957     item.classed("playing", false);
958     d3.select("body").classed("playing", false);
959     item.classed("level-over", true);
960
961     d3.select('meta[property="og:longTitle"]').attr("content", "I've
... made " + dollarFormat0(cash)
962         + " playing the @business Trading Game. Can you beat the
... market? Play now:");
963
964     tickerLabel.style("display", "");
965
966     svg.select(".cursor-indicator")
967         .transition()
968         .attr("r", 1e-6);
969
```

```
970     x.domain(d3.extent(stock.values, f('date')));
971
972     // fed liftoff hack
973     if( d3.max(stock.values, f('price')) > 1000 ) {
974         y.domain(d3.extent(stock.values, f('price')));
975     } else {
976         y.domain([0, d3.max(stock.values, f('price'))]);
977     }
978
979     var transition = svg.transition()
980         .delay(250)
981         .duration(1000);
982
983     transition.select(".x.axis").call(xAxis);
984     transition.select(".y.axis").call(yAxis);
985     transition.select(".gridlines.axis").call(gridlines);
986
987     transition.select("path.line.share")
988         .attr("d", function(d) { return stepLine(stock.values); });
989
990     transition.select("path.area")
991         .attr("d", function(d) { return stepArea(stock.values); });
992
993     transition.selectAll("path.transaction")
994         .attr("d", line);
995
996     transition.select(".y-mark")
997         .attr("transform", "translate(" + (width + 3) + "," +
... y(latest.price) + ")")
998         .style("opacity", 0);
999
1000     // button to advance
1001     toolbar.append("button.next")
1002         .text("Next")
1003         .on("click", getNextLevel)
1004         .on("touchstart", function(d) {
... d3.select(this).classed("hover", true); })
1005         .on("touchend", function(d) { d3.select(this).classed("hover",
... false); });
1006
1007     // update stock properties
1008     stock.cash.push({
1009         "date": d3.max(stock.values, f('date')),
1010         "cash": cash
1011     });
1012     stock.traderReturn = (stock.cash[1].cash - stock.cash[0].cash) /
... stock.cash[0].cash;
1013
1014     if(userHistory[stock.ticker] && stock.traderReturn >
... userHistory[stock.ticker].highscore) {
1015         addAlert("Record personale !", true);
```

```
1016     }
1017
1018     if(stock.competitor && stock.competitor.returns.length === 1) {
1019         if(stock.traderReturn > stock.competitor.returns[0]) {
1020             addAlert("You beat your challenger " +
1021 ... percentFormat0(stock.traderReturn) + " to " +
1022 ... percentFormat0(stock.competitor.returns[0]) + "!", true);
1023         } else {
1024             addAlert("You lost to your challenger, " +
1025 ... percentFormat0(stock.traderReturn) + " to " +
1026 ... percentFormat0(stock.competitor.returns[0]) + ".", true);
1027         }
1028     }
1029
1030     // update user history
1031     if(userHistory[stock.ticker]) {
1032         userHistory[stock.ticker].attempts++;
1033         userHistory[stock.ticker].highscore =
1034 ... Math.max(userHistory[stock.ticker].highscore, stock.traderReturn)
1035     } else {
1036         userHistory[stock.ticker] = {
1037             "attempts": 1,
1038             "highscore": stock.traderReturn
1039         }
1040     }
1041
1042     // - adesso - (numeri puri)
1043     titoliRend.push(stock.traderReturn); // 0.352
1044     sp500Rend.push(stock.indexReturn); // -0.077
1045     var alertText = "<p>La tua strategia ha reso il <strong>" +
1046 ... percentFormat(stock.traderReturn) + "</strong></p>" +
1047     "<p>La strategia BUY&HOLD del titolo ha reso il <strong>" +
1048 ... percentFormat(stock.stockReturn) + "</strong></p>" +
1049     "<p>La strategia BUY&HOLD dell'S&P500 ha reso il <strong>" +
1050 ... percentFormat(stock.indexReturn) + "</strong></p>";
1051
1052     addAlert(alertText, true);
1053
1054     // Commenta questo blocco per disabilitare il salvataggio
1055     /*
1056     if(gameConfig.playable) {
1057         saveTrades(stock, function(err, key) {
1058
1059             stock.challengeUrl = window.location.protocol
1060                 + "://"
1061                 + window.location.host
1062                 + window.location.pathname
1063                 + '?ticker='+key.ticker + '&gameid=' + key.timestamp;
1064
1065             // history.pushState(null, null, stock.challengeUrl);
1066         });
1067     }
1068     */
1069 }
```

```
1059     var alertText = 'Challenge someone to beat your score with
... this URL: <input value="" + stock.challengeUrl + ">';
1060     addAlert(alertText, true);
1061     });
1062   }
1063   */
1064   getTrades(stock.ticker, undefined, renderTrades);
1065
1066   function renderTrades(err, result) {
1067
1068     if(err) {
1069       console.log(err);
1070       return false;
1071     }
1072
1073     if(window.innerWidth > 800 && !gameConfig.ghost) {
1074       svg.selectAll("path.transaction.past")
1075         .data(result.trades)
1076         .enter()
1077         .append("path.transaction.past")
1078         .attr("d", line)
1079         .style("stroke", function(d) { return
... colorScale(d[1].price - d[0].price); })
1080         .style("opacity", 0)
1081         .transition()
1082         .duration(1000)
1083         .delay(1000)
1084         .style("opacity", .05);
1085     }
1086
1087     result.returns.sort(function(a,b){return a - b});
1088     var percentile = result.returns.length ?
... (d3.bisect(result.returns, stock.traderReturn) /
... result.returns.length) : 1;
1089     var rank = result.returns.length - d3.bisect(result.returns,
... stock.traderReturn) + 1;
1090
1091     var performanceComments = [
1092       [0, "Being so bad takes skill."],
1093       [.25, "I suck!"],
1094       [.4, "That's, like, almost average?"],
1095       [.45, "Perfectly average is a kind of perfect."],
1096       [.55, "Above average!"],
1097       [.8, "I should do this for real!"],
1098       [.9, "Wicked hella good!"],
1099       [.99, "Holy shit!!"]
1100   ];
1101     var performanceCommentsScale = d3.scale.threshold()
1102       .domain(performanceComments.map(f(0)).slice(1))
1103       .range(performanceComments.map(f(1)));
1104
```

```
1105     var alertText = "You outperformed " +
...   percentFormat(percentile)
1106     + " of players, ranking #" + rank + ". <span
...   class='tweet'>Tweet</span>";
1107     var tweetText = "I beat " + percentFormat0(percentile)
1108     + " of people trading $" + stock.ticker.toUpperCase()
1109     + ", ranking #" + rank + ". "
1110     + performanceCommentsScale(percentile) + " The Trading
...   Game:";
1111     addAlert(alertText, true, tweetText);
1112
1113     if(gameConfig.stats) {
1114
1115         var returnsAnnotations = [
1116             {"x": stock.stockReturn, "text":
...   stock.ticker.toUpperCase()},
1117             {"x": stock.indexReturn, "text": "S&P"}
1118         ];
1119
1120         if(gameConfig.playable) {
1121             returnsAnnotations.push({"x": stock.traderReturn, "text":
...   "You"});
1122         }
1123
1124         if(stock.competitor && stock.competitor.returns.length ===
...   1) {
1125             returnsAnnotations.push({"x": stock.competitor.returns[0],
...   "text": "Challenger"});
1126         }
1127
1128         // console.log(stock.ticker + ': ' +
...   JSON.stringify(result.returns.map(function(d) { return
...   d.toPrecision(4); })));
1129
1130         if(window.innerWidth > 500) {
1131             var returnsHistogram = histogram()
1132                 .width(300)
1133                 .height(75)
1134                 .title('Returns')
1135                 .annotations(returnsAnnotations);
1136             svg.append("g.histogram.returns")
1137                 .attr("transform", "translate(" + (stock.stockReturn >
...   -.2 ? 20 : (width-320)) + ",10)")
1138                 .datum(result.returns)
1139                 .call(returnsHistogram);
1140
1141             var tradeHistogram = histogram()
1142                 .x(x)
1143                 .height(100);
1144             svg.append("g.histogram.trades")
1145                 .attr("transform", "translate(0," + (height-100) + ")")
```



```
1146         .datum([]).concat.apply([],  
... result.trades).map(f('date'))  
1147         .call(tradeHistogram);  
1148     }  
1149  
1150     if(gameConfig.logging) {  
1151         analysisStats.push({  
1152             "stock ticker": stock.ticker,  
1153             "start date": x.domain()[0],  
1154             "end date": x.domain()[1],  
1155             "stock return": stock.stockReturn,  
1156             "stock deviation": getStdevProper(stock.values),  
1157             "returns mean": d3.mean(result.returns),  
1158             "returns deviation": d3.deviation(result.returns),  
1159             "trades number": result.trades.length,  
1160             "trades duration": d3.mean(result.trades.map(function(d)  
... { return (d[1].date - d[0].date) / (x.domain()[1] - x.domain()[0])  
... })),  
1161             "stock percentile": d3.bisect(result.returns,  
... stock.stockReturn) / result.returns.length,  
1162             "cash percentile": d3.bisect(result.returns, 0) /  
... result.returns.length  
1163         });  
1164     }  
1165  
1166     }  
1167  
1168     if(gameConfig.autoplay) getNextLevel();  
1169 }  
1170  
1171     return true;  
1172 }  
1173  
1174     function gridlineTicks(d) {  
1175         var gridTicks = [];  
1176         var gridInterval = 10;  
1177         for(var i = Math.ceil(y.domain()[0]/gridInterval); i <=  
... Math.floor(y.domain()[1]/gridInterval); i++) {  
1178             gridTicks.push(i*gridInterval);  
1179         }  
1180         return gridTicks;  
1181     }  
1182  
1183     function addAlert(string, stick, tweet) {  
1184         var alert = alerts.append("div.alert")  
1185             .html(string);  
1186  
1187         if(!stick) {  
1188             alert.transition()  
1189                 .delay(4000)  
1190                 .duration(500)
```

```
1191         .remove();
1192     }
1193
1194     if(tweet) {
1195         alert.classed("tweetable", true)
1196         .on("click", function() { postToTwitter(stock.challengeUrl,
... tweet); });
1197     }
1198
1199     alert.select("input").on("click", function() {
1200         this.setSelectionRange(0, this.value.length);
1201     })
1202 }
1203
1204 function showTip() {
1205     if(tips.length) addAlert(tips.shift());
1206 }
1207
1208 });
1209 }
1210
1211 /*  ——— GAME-OVER  ———
1212     Mostra riepilogo rendimenti + pulsante "Gioca ancora"
1213 */
1214 function gameOver () {
1215
1216     /*-- ri-mostra l'header --*/
1217     d3.select('#full-header').classed('hidden', false);
1218
1219     /*-- overlay centrale --*/
1220     const overlay = d3.select('.levels')
1221         .append('div')
1222         .attr('class', 'game-over')
1223         .style('z-index', d3.selectAll('.levels > div').size());    //
... sopra a tutto
1224
1225     const center = overlay.append('div').attr('class', 'center-middle');
1226
1227
1228     /*—————
1229         1.  RIEPILOGO DEI RENDIMENTI
1230     —————*/
1231
1232     const fmt = d3.format('.1%');
1233
1234
1235
1236     const righe = titoliRend.map((r, i) =>
1237         `Titolo ${i + 1} tuo rendimento: ${fmt(r)}`);    // ← ora r è un
... numero
1238
```

```
1239     const mediaTitoli = d3.mean(titoliRend);    // funziona perché sono
... numeri
1240     const mediaSp500  = d3.mean(sp500Rend);
1241
1242     righe.push(`Media tuo rendimento titoli: ${fmt(mediaTitoli)}`);
1243     righe.push(`Media rendimento S&P500: ${fmt(mediaSp500)}`);
1244
1245
1246
1247     /*-- paragrafo riepilogo --*/
1248     center.append('p')
1249       .html(righe.join('<br>'))
1250       .style('font-size', '2em')
1251       .style('margin-bottom', '20px')
1252       .style('text-align', 'left');
1253
1254
1255     /*-----
1256     2.  TITOLONE + PULSANTE "GIOCA ANCORA"
1257     -----*/
1258
1259     center.append('h2').text('GAME OVER');
1260
1261     center.append('button')
1262       .style('background', '#5bc0ff')    // azzurro
1263       .style('color',      '#00334d')    // testo blu scuro (contrasto)
1264       .text('Gioca ancora')
1265       .on('click', () => location.reload());
1266
1267
1268     /*-----
1269     3.  CANVAS DECORATIVO (come versione originale)
1270     -----*/
1271     initEnderCanvas(overlay.append('canvas'));
1272
1273
1274     /*-----
1275     4.  LOG DI DEBUG (se abilitato)
1276     -----*/
1277     if (gameConfig.logging) {
1278       console.table(analysisStats);
1279       console.table(cashRules);
1280     }
1281   }
1282
1283
1284   ///////////////////////////////////
1285   // HELPERS AND STUFF
1286
1287   function getSmoothDomainFunction(data) {
1288
```

```
1289 // simplify price
1290 // tolerance comes from price
1291 var dataSimple = simplify(data.map(function(d) {
1292     return {
1293         "x": +d.date,
1294         "y": d.price
1295     }
1296 })), d3.mean(data.map(f('σ')))*1.5);
1297 var dataSimpleLine = dataSimple.map(function(d) {
1298     return {
1299         "date": new Date(d.x),
1300         "price": d.y
1301     }
1302 });
1303 // smooth price
1304 var smoothPath = Smooth(dataSimple.map(function(d) {
1305     return [d.x, d.y];
1306 })));
1307
1308 // simply variance
1309 // tolerance is just... high. too much volatility in aperture is
... disorienting.
1310 var dataStdevSimple = simplify(data.map(function(d) {
1311     return {
1312         "x": +d.date,
1313         "y": d.σ
1314     }
1315 })), 30);
1316 var dataStdevSimpleLine = dataStdevSimple.map(function(d) {
1317     return {
1318         "date": new Date(d.x),
1319         "price": d.y
1320     }
1321 });
1322 // smooth variance
1323 var smoothStdevPath = Smooth(dataStdevSimple.map(function(d) {
1324     return [d.x, d.y];
1325 })));
1326
1327 // example usage in situ: getSmoothDomain(tScale.invert(t));
1328 function getSmoothDomain(date) {
1329     // SMOOTH AND SIMPLIFY BOUNDS STUFF
1330     var bi = d3.bisect(dataSimpleLine.map(f('date')), date);
1331     if(bi < dataSimpleLine.length) {
1332         var biPc = (+date - +dataSimpleLine[bi-1].date) /
... (+dataSimpleLine[bi].date - +dataSimpleLine[bi-1].date)
1333         // console.log("We are " + Math.round(biPc*100) + "% between " +
... dataSimpleLine[bi-1].date + " and " + dataSimpleLine[bi].date);
1334         var smoothPrice = smoothPath((bi-1) + biPc)[1];
1335     } else {
1336         var smoothPrice = smoothPath(bi)[1];
```

```
1337     }
1338     // END SMOOTH AND SIMPLIFY BOUNDS STUFF
1339
1340     // SMOOTH AND SIMPLIFY VARIANCE STUFF
1341     var bi = d3.bisect(dataStdevSimpleLine.map(f('date')), date);
1342     if(bi < dataStdevSimpleLine.length) {
1343         var biPc = (+date - +dataStdevSimpleLine[bi-1].date) /
... (+dataStdevSimpleLine[bi].date - +dataStdevSimpleLine[bi-1].date)
1344         var smoothStdev = smoothStdevPath((bi-1) + biPc)[1];
1345     } else {
1346         var smoothStdev = smoothStdevPath(bi)[1];
1347     }
1348     // END SMOOTH AND SIMPLIFY VARIANCE STUFF
1349
1350     var aperture = smoothPrice * .2 + smoothStdev * 4; //original
1351     var aperture = smoothPrice * 0 + smoothStdev * 5; //for fed
... liftoff
1352
1353     var domain = [
1354         Math.max(smoothPrice - aperture, 0),
1355         smoothPrice + aperture
1356     ];
1357
1358     domain.price = smoothPrice;
1359     domain.stdev = smoothStdev;
1360
1361     return domain;
1362 }
1363
1364 getSmoothDomain.price = dataSimpleLine;
1365 getSmoothDomain.stdev = dataStdevSimpleLine;
1366
1367 return getSmoothDomain;
1368 }
1369
1370 function getStdev(d) {
1371     // compute from periodicity of data
1372     var period = Math.min(this[1].date - this[0].date, this[2].date -
... this[1].date);
1373     var varianceWindow = period * 90;
1374
1375     // N.B. #BUG:
1376     // IF PERIODICITY IS SUFFICIENTLY BIMODAL,
1377     // there can be gaps greater than the varianceWindow,
1378     // which leads to this returning 'undefined' stdev,
1379     // which makes it unreliable when fed into smooth+simplify
1380     // for stocks with huge gaps relative to the granularity
1381     // (e.g. a week of 5-minute data w/ nights and weekends)
1382
1383     var bounds = [
1384         new Date(+d.date - (.5 * varianceWindow)),
```

```
1385     new Date(+d.date + (.5 * varianceWindow))
1386 ];
1387 var boundedData = this.filter(function(dd) { return dd.date >
... bounds[0] && dd.date <= bounds[1]; });
1388
1389 d.σ = d3.deviation(boundedData, f('price'));
1390 }
1391
1392 // calculate stdev from intraday returns
1393 //
... http://www.investopedia.com/ask/answers/021015/how-can-you-calculate-
... volatility-excel.asp
1394 function getStdevProper(values) {
1395     values.forEach(function(d, i) {
1396         if(i === 0) return;
1397         d.return = (d.price / values[i-1].price) - 1;
1398     })
1399     return d3.deviation(values, f('return'));
1400 }
1401
1402 function getStdev2(d) {
1403     // compute from periodicity of data
1404     var period = Math.min(this[1].date - this[0].date, this[2].date -
... this[1].date);
1405     var varianceWindow = period * 90;
1406
1407     // N.B. #BUG:
1408     // IF PERIODICITY IS SUFFICIENTLY BIMODAL,
1409     // there can be gaps greater than the varianceWindow,
1410     // which leads to this returning 'undefined' stdev,
1411     // which makes it unreliable when fed into smooth+simplify
1412     // for stocks with huge gaps relative to the granularity
1413     // (e.g. a week of 5-minute data w/ nights and weekends)
1414
1415     var bounds = [
1416         new Date(+d.date - (.5 * varianceWindow)),
1417         new Date(+d.date + (.5 * varianceWindow))
1418     ];
1419     var boundedData = this.filter(function(dd) { return dd.date >
... bounds[0] && dd.date <= bounds[1]; });
1420
1421     boundedData.forEach(function(d, i) {
1422         if(i === 0) return;
1423         d.return = (d.price / boundedData[i-1].price) - 1;
1424     })
1425
1426     d.σ2 = d3.deviation(boundedData, f('return'));
1427 }
1428
1429 function timerFormat(t) {
1430     var secondsFormat = d3.format('02');
```

```
1431     return (Math.ceil(t/1000) == 60 ? "1" : "") + ":" +
... secondsFormat(Math.abs(Math.ceil(t/1000) % 60));
1432 }
1433
1434 // https://css-tricks.com/snippets/javascript/get-url-variables/
1435 function getQueryVariable(variable) {
1436     var query = window.location.search.substring(1);
1437     var vars = query.split("&");
1438     for (var i=0;i<vars.length;i++) {
1439         var pair = vars[i].split("=");
1440         if(pair[0] == variable){return pair[1];}
1441     }
1442     return(false);
1443 }
1444
1445 ///////////////
1446 // CANVAS
1447
1448 function initCanvas() {
1449     var canvas = d3.select(".opener canvas");
1450     canvas.node().setAttribute('width', innerWidth);
1451     canvas.node().setAttribute('height', innerHeight);
1452     var ctx = canvas.node().getContext('2d');
1453
1454     initStocks();
1455
1456     function initStocks() {
1457         // n.b. hella clever random color
... http://www.paulirish.com/2009/random-hex-color-code-snippets/
1458         var stocks = d3.range(20).map(function(d) {
1459             return {
1460                 "t": 0,
1461                 "p": innerHeight * Math.random(),
1462                 "color": '#' + Math.floor(Math.random()*16777215).toString(16)
1463             };
1464         });
1465         d3.timer(function(t) { return renderCanvas(t, stocks); });
1466     }
1467
1468     function getPriceChange(stock) {
1469         var dp = (d3.random.logNormal(.5,1)()) * -1 + 3) / 200;
1470         stock.p = stock.p + stock.p * dp;
1471     }
1472
1473     function renderCanvas(t, stocks) {
1474
1475         // if user has advanced, stop animation
1476         if(levelNumber > 0) return true;
1477
1478         // if time is up, restart timer
1479         if(t/20 > innerWidth) {
```

```
1480     initStocks();
1481     return true;
1482 }
1483
1484 stocks.forEach(function(d,i) {
1485
1486     ctx.strokeStyle = d.color;
1487
1488     if(d.p >= innerHeight) {
1489         d.p = innerHeight * Math.random();
1490         return;
1491     }
1492
1493     ctx.beginPath();
1494     ctx.moveTo(d.t/20, innerHeight - d.p);
1495
1496     getPriceChange(d);
1497     d.t = t % (innerWidth*20);
1498
1499     ctx.lineTo(d.t/20, innerHeight - d.p);
1500     ctx.stroke();
1501
1502 })
1503 }
1504 }
1505
1506 function initEnderCanvas(canvas) {
1507     var width = innerWidth;
1508     var height = innerHeight;
1509     canvas.node().setAttribute('width', width);
1510     canvas.node().setAttribute('height', height);
1511     var ctx = canvas.node().getContext('2d');
1512
1513     initStocks();
1514
1515     function initStocks() {
1516         // n.b. hella clever random color
1517         ... http://www.paulirish.com/2009/random-hex-color-code-snippets/
1518         var stocks = d3.range(20).map(function(d) {
1519             return {
1520                 "t": 0,
1521                 "p": Math.min(innerHeight, innerWidth) * Math.random() * .7,
1522                 "offset": Math.PI * 2 * Math.random(),
1523                 "color": '#' + Math.floor(Math.random()*16777215).toString(16)
1524             };
1525         });
1526         d3.timer(function(t) { return renderCanvas(t, stocks); });
1527     }
1528
1529     function getPriceChange(stock) {
1530         var dp = (d3.random.logNormal(.5,1)() * -1 + 3) / 200;
```



```
1530     stock.p = stock.p + stock.p * dp;
1531 }
1532
1533 function renderCanvas(t, stocks) {
1534
1535     var timeToAngle = d3.scale.linear()
1536         .domain([0,5000])
1537         .range([0,Math.PI*2]);
1538
1539     // if time is up, restart timer
1540     if(t/20 > innerWidth) {
1541         initStocks();
1542         return true;
1543     }
1544
1545     stocks.forEach(function(d,i) {
1546
1547         ctx.strokeStyle = d.color;
1548
1549         // get r and theta from d.p and t
1550         var angle = timeToAngle(d.t) + d.offset;
1551         var radius = d.p;
1552         var cartX = radius * Math.cos(angle) + width/2;
1553         var cartY = radius * Math.sin(angle) + height/2;
1554
1555         ctx.beginPath();
1556         ctx.moveTo(cartX, cartY);
1557
1558         getPriceChange(d);
1559         d.t = t;
1560
1561         var angle = timeToAngle(d.t) + d.offset;
1562         var radius = d.p;
1563         var cartX = radius * Math.cos(angle) + width/2;
1564         var cartY = radius * Math.sin(angle) + height/2;
1565
1566         ctx.lineTo(cartX, cartY);
1567         ctx.stroke();
1568
1569     })
1570 }
1571 }
1572
1573 ///////
1574 // ADS
1575
1576 function getBestAdSize(container) {
1577     /*
1578     IAB ad unit nomenclature:
1579     -----
1580     300 × 250    Medium Rectangle
```

```
1581 180 × 150   Rectangle
1582 160 × 600   Wide Skyscraper
1583 728 × 90    Leaderboard
1584 -----
1585 http://www.iab.net/guidelines/508676/508767/UAP
1586 */
1587 var sizes = [[300,250],[180,150],[160,600],[728,90]];
1588 var eligibleSizes = sizes.filter(function(d) { return d[0] <
... container.offsetWidth; });
1589 var widestSize = _.max(eligibleSizes,function(d) { return d[0]; });
1590 return widestSize;
1591 }
1592
1593 //////////
1594 // SERVER
1595
1596 var lambda = false;
1597 if(typeof AWS !== 'undefined' && !isTerminal) {
1598   lambda = new AWS.Lambda({
1599     region: "us-east-1",
1600     accessKeyId: "AKIAJG4MLF2YGY5XX7UA",
1601     secretAccessKey: "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"
1602   });
1603 }
1604
1605 function saveTrades(stock, callback) {
1606
1607   if(!lambda) {
1608     if(callback) callback.call(this, new Error("Lambda not defined;
... AWS failed us!"));
1609     return false;
1610   }
1611
1612   var newItem = {
1613     "ticker": stock.ticker,
1614     "timestamp": +new Date(),
1615     "user": userId,
1616     "screen": [innerWidth, innerHeight],
1617     "competitor": stock.competitor ? stock.competitor.gameid : false,
1618     "attempts": userHistory[stock.ticker] ?
... userHistory[stock.ticker].attempts : false,
1619     "return": stock.traderReturn, // ".217"
1620     "cash": stock.cash, // [{date, cash}, {date, cash}]
1621     "trades": stock.trades // jsonification drops the .outstanding and
... .id array object props
1622   }
1623
1624   var params = {
1625     "operation": "create",
1626     "Item": newItem
1627   }
```

```
1628
1629     lambda.invoke({
1630         FunctionName: gameConfig.saveServer,
1631         Payload: JSON.stringify(params)
1632     }, function(err, data) {
1633         if (!err) {
1634             // result is currently an empty object, but hey
1635             var result = JSON.parse(data.Payload)
1636
1637             // return a key to the thing just inserted
1638             var newKey = {
1639                 "ticker": newItem.ticker,
1640                 "timestamp": newItem.timestamp
1641             };
1642
1643             if(callback) callback.call(this, undefined, newKey);
1644         } else {
1645             console.log("err: " + err);
1646             if(callback) callback.call(this, err, undefined);
1647         }
1648     });
1649 });
1650
1651 }
1652
1653 function getTrades(ticker, timestamp, callback) {
1654     if(timestamp === undefined) {
1655         // get cached crowd trades, if available; fail over to AWS Lambda
1656         // Dynamo
1657         d3.json("data/leaderboards/" + ticker + ".json", function(error,
1658             data) {
1659             if(error || gameConfig.alwaysUseServer) {
1660                 getTradesServer(ticker, undefined, callback);
1661             } else {
1662                 data.trades.forEach(function(pair) {
1663                     pair.forEach(function(endpoint) {
1664                         endpoint.price = +endpoint.price;
1665                         endpoint.date = new Date(endpoint.date);
1666                     });
1667                 });
1668             }
1669             callback(error, data);
1670         });
1671     } else {
1672         // can't use cached when we're looking for a specific timestamp
1673         getTradesServer(ticker, timestamp, callback);
1674     }
1675 }
1676
1677 function getTradesServer(ticker, timestamp, callback) {
1678     var params = {
```

```
1677     operation: "list",
1678     ticker: ticker,
1679     timestamp: timestamp,
1680     limit: gameConfig.readServerLimit
1681 }
1682
1683 paginateLambdaRequest([], params, function(err,data) {
1684
1685     if(gameConfig.logging) {
1686         cashRules.push(d3.sum(data.map(f('cash','1','cash'))));
1687     }
1688
1689     var returnData = {
1690         "returns": data.map(f('return')),
1691         "trades": [].concat.apply([], data.map(f('trades')))
1692         // "nestedTrades": data.map(f('trades'))
1693     };
1694
1695     returnData.trades.forEach(function(pair) {
1696         pair.forEach(function(endpoint) {
1697             endpoint.price = +endpoint.price;
1698             endpoint.date = new Date(endpoint.date);
1699         })
1700     });
1701
1702     console.log("$" + ticker + ": " + returnData.trades.length + "
... trades from " + returnData.returns.length + " playthroughs.");
1703     // console.log("//// ALL DATA FOR " + ticker.toUpperCase() + "
... ////");
1704     // console.log(JSON.stringify(data));
1705     // console.log("//// COMPRESSED ////");
1706     // console.log(JSON.stringify(returnData));
1707     // console.log("//// END DATA FOR " + ticker.toUpperCase() + "
... ////");
1708
1709     callback(err, returnData);
1710 })
1711 }
1712
1713 function paginateLambdaRequest(results, params, callback) {
1714
1715     if(!lambda) {
1716         if(callback) callback.call(this, new Error("Lambda not defined;
... AWS failed us!"));
1717         return false;
1718     }
1719
1720     // console.log("requesting with", JSON.stringify(params));
1721
1722     lambda.invoke({
1723         FunctionName: gameConfig.readServer,
```

```
1724     Payload: JSON.stringify(params)
1725 }, function(err, data) {
1726
1727     if (!err) {
1728         data = JSON.parse(data.Payload)
1729         results = results.concat(data.Items);
1730
1731         console.log(results.length, data.Count, params.limit,
... data.LastEvaluatedKey);
1732
1733         if(data.LastEvaluatedKey !== undefined && (params.limit ===
... undefined || data.Count < params.limit)) {
1734             if(params.limit) params.limit -= data.Count;
1735             params.startKey = data.LastEvaluatedKey;
1736             paginateLambdaRequest(results, params, callback);
1737         } else {
1738             callback(err, results);
1739         }
1740
1741     } else {
1742         console.log("err: " + err);
1743         callback(err, results);
1744     }
1745
1746 });
1747
1748 }
1749
1750 function closest(accessor) {
1751     var bi = d3.bisector(accessor).right;
1752     return function(array, item) {
1753         var i = bi(array, item);
1754         var left = array[i-1];
1755         var right = array[i];
1756
1757         var dLeft = (left !== undefined) ? Math.abs(accessor(left) - item)
... : Infinity;
1758         var dRight = (right !== undefined) ? Math.abs(accessor(right) -
... item) : Infinity;
1759
1760         return dLeft <= dRight ? left : right;
1761     }
1762 }
1763
1764 function centerPopup(width, height) {
1765     var wLeft = window.screenLeft ? window.screenLeft : window.screenX;
1766     var wTop = window.screenTop ? window.screenTop : window.screenY;
1767     var left = wLeft + (window.innerWidth / 2) - (width / 2);
1768     var top = wTop + (window.innerHeight / 2) - (height / 2);
1769     return 'width=' + width + ',height=' + height + ',top=' + top +
... ',left=' + left;
```

```
1770 }
1771
1772
1773
1774 //////////
1775 // STATS
1776
1777 // based on http://bl.ocks.org/mbostock/3048450
1778 function histogram() {
1779
1780     var width = 960,
1781         height = 500,
1782         x,
1783         title,
1784         annotations = [];
1785
1786     function chart(selection) {
1787         selection.each(function(values) {
1788
1789             if(!values || !values.length) return;
1790
1791             if(x === undefined) {
1792                 // Either fit scale to middle 98% of values...
1793                 values = values.filter(function(d) {
1794                     return d >= d3.quantile(values,.01) && d <
1795 ... d3.quantile(values,.99);
1796                 })
1797                 x = d3.scale.linear()
1798                     .domain(d3.extent(values))
1799                     .range([0, width]);
1800             } else {
1801                 // ...or fit values to scale.
1802                 values = values.filter(function(d) {
1803                     return d >= x.domain()[0] && d < x.domain()[1];
1804                 })
1805             }
1806
1807             // A formatter for counts.
1808             var formatCount = d3.format(",.0f");
1809
1810             // Use date scale for dates, percentage scale otherwise. Sloppy,
1811 ... yes.
1812             if(values[0] instanceof Date) {
1813                 var formatLabel = function(d) {
1814                     var extent = d3.extent(values)[1] - d3.extent(values)[0];
1815                     var format = d3.time.format.multi([
1816                         [".%L", function(d) { return extent < 1000; }],
1817                         [":%S", function(d) { return extent < 1000 * 60; }],
1818                         ["%I:%M", function(d) { return extent < 1000 * 60 * 60; }],
1819 ... ]],
1820                 ["%I %p", function(d) { return extent < 1000 * 60 * 60 *
```

```
1817... 24; }],
1818         ["%a %d", function(d) { return extent < 1000 * 60 * 60 *
... 24 * 7; }],
1819         ["%b %d", function(d) { return extent < 1000 * 60 * 60 *
... 24 * 31; }],
1820         ["%B", function(d) { return extent < 1000 * 60 * 60 *
... 24 * 365; }],
1821         ["%Y", function() { return true; }]
1822     ]);
1823     return format(new Date(d));
1824 }
1825 } else {
1826     var formatLabel = d3.format(".0%");
1827 }
1828
1829 // Generate a histogram using twenty uniformly-spaced bins.
1830 var data = d3.layout.histogram()
1831     .bins(75)
1832     (values);
1833
1834 var y = d3.scale.linear()
1835     .domain([0, d3.max(data, function(d) { return d.y; })])
1836     .range([height, 0]);
1837
1838 var xAxis = d3.svg.axis()
1839     .scale(x)
1840     .orient("bottom")
1841     .tickFormat(formatLabel);
1842
1843 var svg = d3.select(this);
1844
1845 var titleText = svg.selectAll("text.title").data([title]);
1846 titleText.enter().append("text.title");
1847 titleText.text(f());
1848
1849 var bar = svg.selectAll(".bar")
1850     .data(data);
1851
1852 bar.exit().remove();
1853
1854 var barEnter = bar.enter().append("g")
1855     .attr("class", "bar");
1856
1857 svg.selectAll(".bar")
1858     .attr("transform", function(d) { return "translate(" +
... x(d.x) + "," + y(d.y) + ")"; });
1859
1860 barEnter.append("rect")
1861     .attr("x", 1);
1862
1863 svg.selectAll(".bar").select("rect")
```

```
1864         .attr("width", x(data[1].x) - x(data[0].x))
1865         .attr("height", function(d) { return height - y(d.y); });
1866
1867     // barEnter.append("text")
1868     //     .attr("dy", ".75em")
1869     //     .attr("y", 6)
1870     //     .attr("x", (x(data[1].x) - x(data[0].x)) / 2)
1871     //     .attr("text-anchor", "middle");
1872
1873     // svg.selectAll(".bar").select("text")
1874     //     .text(function(d) { return formatCount(d.y); });
1875
1876     svg.selectAll(".x.axis")
1877         .data([data])
1878         .enter().append("g")
1879         .attr("class", "x axis");
1880
1881     svg.selectAll(".x.axis")
1882         .attr("transform", "translate(0," + height + ")")
1883         .call(xAxis);
1884
1885     var annos = svg.selectAll("g.annotation")
1886         .data(annotations);
1887     var annoEnter = annos.enter().append("g.annotation");
1888     annoEnter.append("line");
1889     annoEnter.append("text");
1890     annos
1891         .attr("transform", function(d) { return "translate(" + x(d.x)
... + ",0)"; })
1892     annos.select("line")
1893         .attr("x1", 0)
1894         .attr("x2", 0)
1895         .attr("y1", function(d,i) { return .3*height + i * 12})
1896         .attr("y2", height);
1897     annos.select("text")
1898         .text(f('text'))
1899         .attr("y", function(d,i) { return .3*height + i * 12})
1900         .attr("dy", " -.5em");
1901
1902     })
1903 }
1904
1905 chart.width = function(_) {
1906     if (!arguments.length) return width;
1907     width = _;
1908     return chart;
1909 };
1910
1911 chart.height = function(_) {
1912     if (!arguments.length) return height;
1913     height = _;
```



```
1914     return chart;
1915 };
1916
1917 chart.x = function(_) {
1918     if (!arguments.length) return x;
1919     x = _;
1920     return chart;
1921 };
1922
1923 chart.title = function(_) {
1924     if (!arguments.length) return title;
1925     title = _;
1926     return chart;
1927 };
1928
1929 chart.annotations = function(_) {
1930     if (!arguments.length) return annotations;
1931     _.sort(function(a,b) { return b.x - a.x; });
1932     annotations = _;
1933     return chart;
1934 };
1935
1936 return chart;
1937 }
1938
1939
```